Data Pipeline Design

什麼是 Data Pipeline？

你的 App 每天都在產生資料：用戶點擊、購買行為、感測器回報、伺服器 log。這些原始資料本身沒什麼用，它們是零散的事件，散落在各個系統裡。真正有價值的，是你能從中提取出來的洞察：哪些功能受歡迎？哪些用戶快要流失？今天的營收比昨天多多少？

Data Pipeline（資料管線） 就是把原始資料從產生的地方，轉化、搬移到能被分析或消費的地方的這一整套系統。它是一條從資料來源（source）到目的地（sink）的自動化流水線。

聽起來簡單，但一旦系統變大，問題接踵而來：資料量大到無法即時處理怎麼辦？某個步驟失敗了，資料會不會重複計算或遺失？上游資料格式改了，下游怎麼辦？這些才是 data pipeline 設計的真正挑戰。

這篇講義涵蓋批次（Batch）與串流（Streaming）兩種處理模式、常見的架構模式（Lambda、Kappa）、核心的容錯機制，以及如何在系統設計面試中有條理地討論這個主題。

bulomoat.org

核心問題：批次還是串流？

設計 data pipeline 的第一個決策，是選擇**處理時機**：你要等資料積累一批再一起處理，還是每筆資料產生後立刻處理？

這不只是技術選擇，它反映了你對業務需求的判斷。

Batch Processing（批次處理）

批次處理把資料分成一批一批地處理，通常是固定時間間隔（每小時、每天）或累積到一定量後觸發。它處理的是「靜止的資料」（data at rest），資料先存起來，等到時機到了再一起跑。

適合批次處理的場景：

- **報表與分析**：每天早上 6 點跑昨天的銷售報告，不需要即時性。

- **大規模資料轉換**：把整個月的用戶行為 log 轉換成 training data，資料量太大，必須分批處理。
- **機器學習訓練**：用過去 90 天的資料訓練推薦模型，本來就是在歷史資料上跑。
- **帳單計算**：每月月底算每個客戶的使用量，精確性比速度更重要。

批次處理的優點是**吞吐量高**、**成本低**。你可以在離峰時段把算力砸進去，充分利用資源。缺點是**延遲高，因為**資料從產生到可以被用，至少要等到下一個批次跑完。

Stream Processing（串流處理）

串流處理把每一筆進來的資料視為事件，產生後立刻處理。它處理的是「移動中的資料」（data in motion），過程中不積累、不等待，來一筆處理一筆。

適合串流處理的場景：

- **詐欺偵測**：信用卡刷卡當下就要判斷是否異常，等批次跑完早就太晚了。
- **即時儀表板**：展示「過去 5 分鐘的訂單數」，需要持續更新的聚合值。
- **個人化推薦**：用戶剛看完一部電影，下一秒就要更新他的推薦列表。
- **異常告警**：伺服器 CPU 飆升，應該立刻觸發告警，不是等晚上的報告才發現。

串流處理的優點是**低延遲**，資料產生後幾秒甚至幾毫秒內就能得到結果。缺點是**複雜度高**：處理失敗怎麼重試、如何保證不重複計算、亂序到達的事件怎麼辦，這些問題在批次處理裡幾乎不存在，在串流處理裡卻都要解決。

buildnoat.org

批次處理的技術棧

MapReduce 的思想

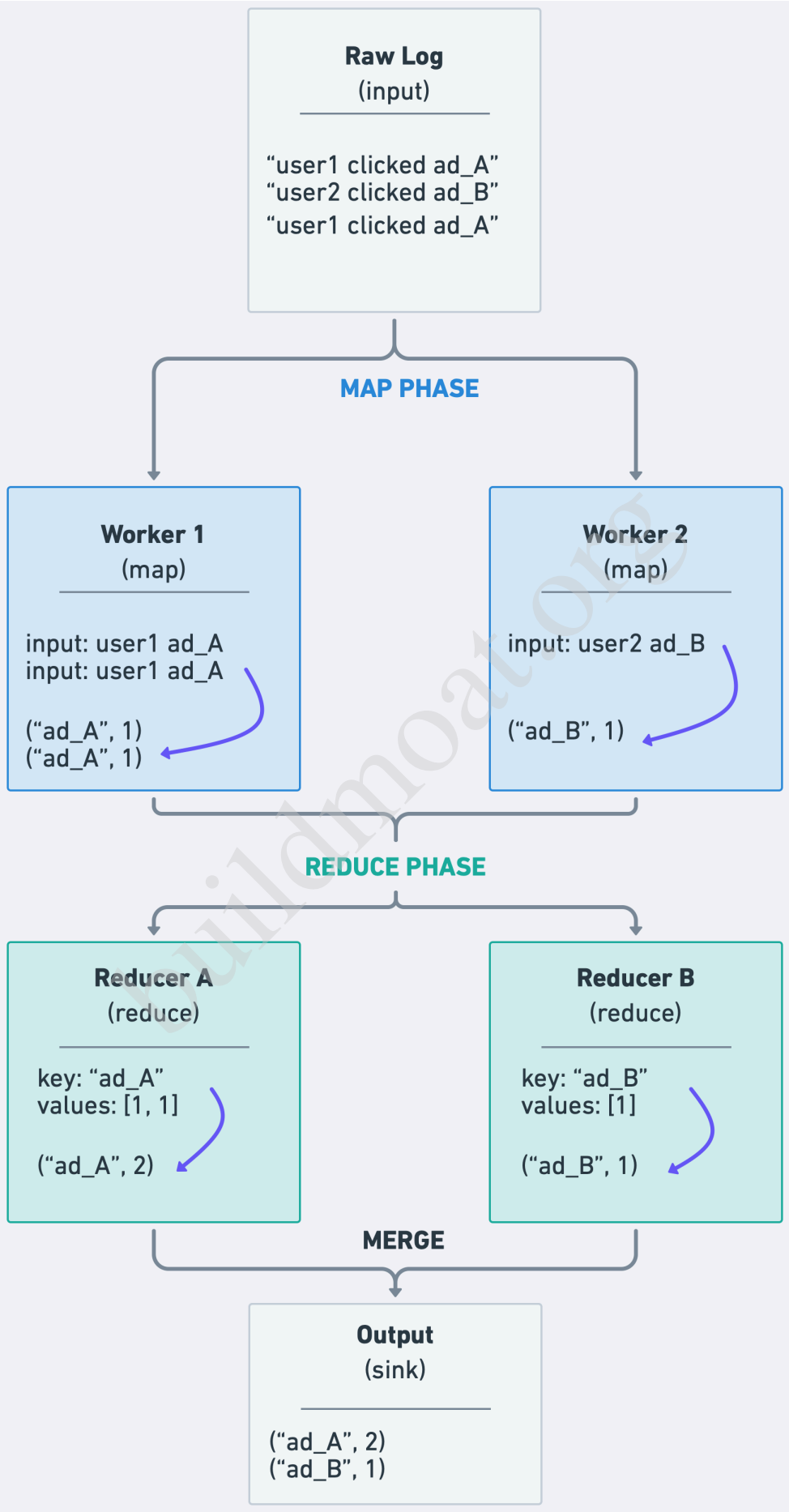
理解批次處理，要從 **MapReduce** 這個概念開始。它是 Google 在 2004 年提出的分散式計算模型，影響了後來幾乎所有的批次處理框架。

核心思想很簡單：把計算拆成兩個階段。

Map 階段：把輸入資料拆成許多小塊，分發給多台機器平行處理，每台機器把資料轉換成 key-value pair。

Reduce 階段：把相同 key 的所有 value 聚合在一起，計算最終結果。

buildmoat.org



buildmoat.org

這個模式的強大之處在於天然的可擴展性：有多少資料，就開多少台 Map worker；需要多少種聚合，就開多少 Reducer。單台機器解決不了的問題，幾百台機器一起上。

Apache Spark

現代批次處理的主流框架。Spark 解決了 MapReduce 最大的痛點，就是每個步驟之間都要把資料寫到磁碟，速度很慢。Spark 把中間結果盡量保留在記憶體裡，比 Hadoop MapReduce 快 10 到 100 倍。

Spark 的核心抽象是 RDD（Resilient Distributed Dataset）和更高階的 **DataFrame API**，讓你用類似 SQL 的語法描述計算邏輯，由框架決定怎麼分散執行。

```
# 計算每個廣告的點擊數
df = spark.read.parquet("s3://logs/clicks/2024-01-01/")
result = (df
    .filter(df.event_type == "click")
    .groupBy("ad_id")
    .count()
    .orderBy("count", ascending=False))

result.write.parquet("s3://output/ad_clicks/")
```

面試中你只需要知道：Spark 是批次處理的預設答案。當你需要處理 TB 級以上的歷史資料、跑 ETL、或訓練 ML 模型，說「用 Spark 跑批次」是完全合理的選擇。

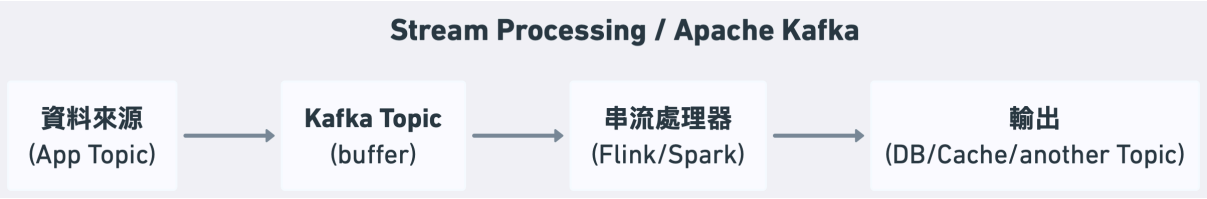
buildmoat.org

串流處理的技術棧

Apache Kafka 作為事件骨幹

串流處理通常以 **Kafka** 作為中心。Kafka 是一個分散式事件串流平台——你可以把它想成一個超高吞吐量的、持久化的訊息 bus。

上游系統（producers）把事件發布到 Kafka topic；下游系統（consumers）訂閱 topic，以自己的速度消費事件。Kafka 把事件保留一段時間（預設 7 天），讓 consumer 可以重播（replay）歷史事件，這在批次和串流處理中都非常有用。



Apache Flink

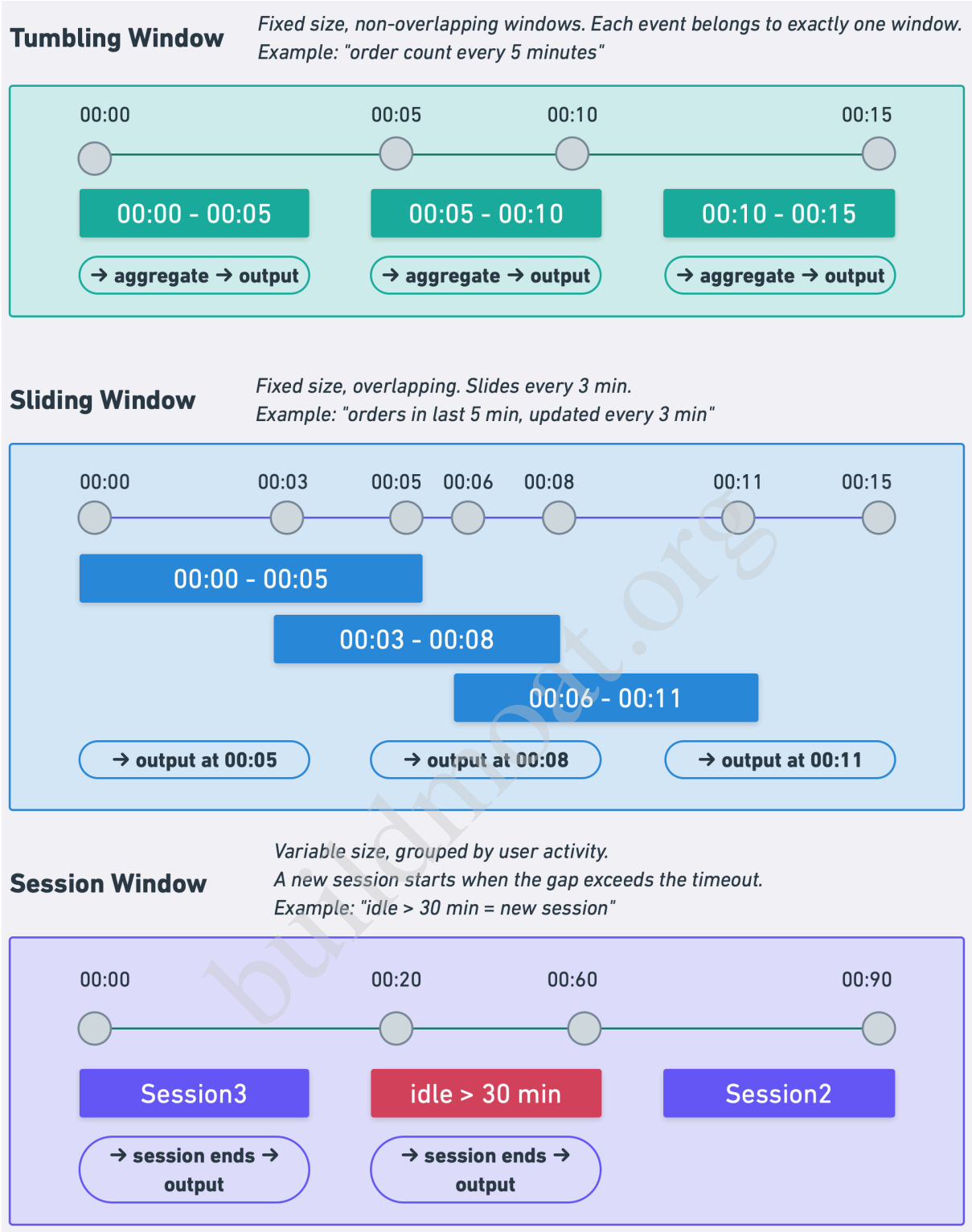
串流處理的現代主流框架，被 Airbnb、Uber、阿里巴巴等大型公司廣泛使用。

Flink 的核心概念：

Window（視窗）：串流處理不能無限等所有資料到齊，必須在某個時間範圍內進行聚合。Flink 支援多種視窗：

- **Tumbling Window（滾動視窗）**：每 5 分鐘一個視窗，不重疊。「過去 5 分鐘的訂單數」。
- **Sliding Window（滑動視窗）**：每 1 分鐘更新一次，但覆蓋過去 5 分鐘。「最近 5 分鐘的訂單數，每分鐘更新」。
- **Session Window（會話視窗）**：按用戶活動分組，閒置超過 30 分鐘就算一個 session 結束。

buildmoat.org



buildmoat.org

Event Time vs Processing Time（事件時間 vs 處理時間）：

這是串流處理最微妙的問題。事件的「發生時間」和「到達處理器的時間」往往不一樣——手機離線時產生的事件，等網路恢復才送到。

如果你用「處理時間」當基準，離線產生的事件可能被分到錯誤的視窗。更好的做法是使用事件時間，也就是用事件本身記錄的真實發生時間，但這樣就需要處理遲到事

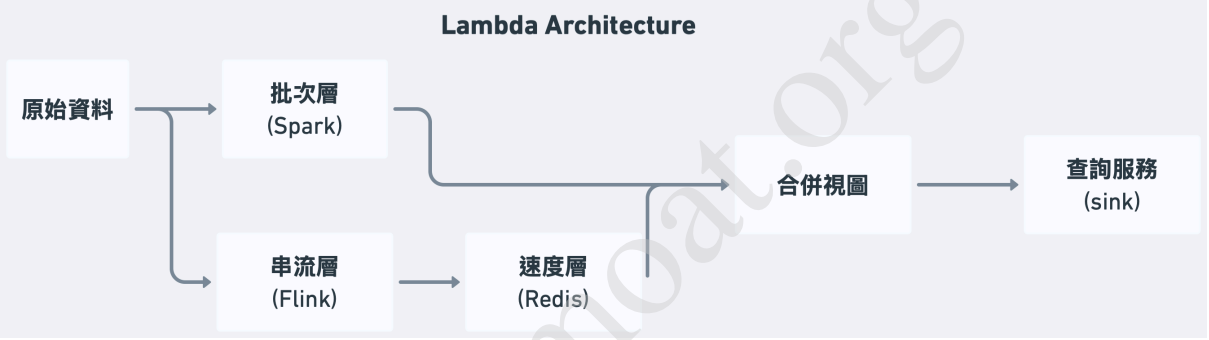
件，那些該屬於已關閉視窗的事件怎麼辦？

Flink 用 **Watermark** 來解決這個問題。Watermark 是處理器對「這個時間點之前的所有事件都已到達」的聲明，一旦 watermark 推進，對應的視窗就可以關閉並輸出結果。合理設定 watermark 延遲（例如允許事件最多遲到 10 秒），能在結果準確性和輸出延遲之間取得平衡。

兩種架構哲學

Lambda Architecture

Lambda Architecture 的核心思想是：**同時跑批次和串流兩條管線，用批次層保證正確性，用串流層保證低延遲，然後把兩者的結果合併。**



三個層次：

Batch Layer（批次層）：定期（例如每小時）對所有歷史資料跑一次完整的計算，把結果存到 Batch View。這個結果是完全正確的，因為它有完整的資料。缺點是延遲高。

Speed Layer（速度層）：只處理最近一段時間（例如最近兩小時）的增量資料，給出低延遲但可能略有誤差的近似結果。

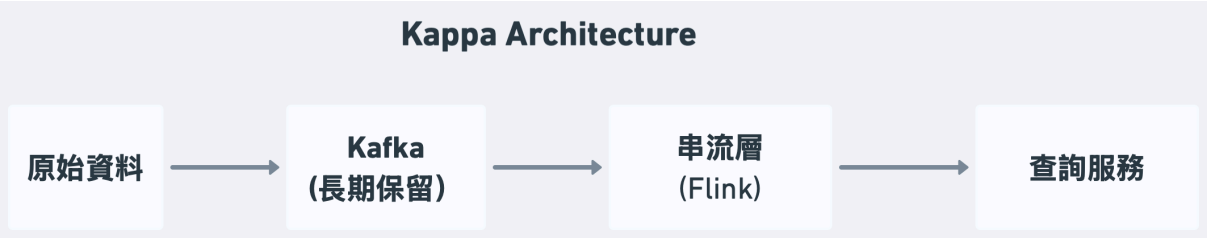
Serving Layer（服務層）：查詢時合併批次層和速度層的結果，用批次層的結果覆蓋速度層中已經被批次計算涵蓋的部分。

Lambda Architecture 的問題：你需要維護兩套程式碼，一套是批次，另一套串流，但它們應該實現相同的業務邏輯。任何邏輯改動，要同步修改兩個地方。這是一個巨大的維護負擔。

Kappa Architecture

Kappa Architecture 是對 Lambda 的反思：**既然串流處理可以重播歷史資料，為什麼要維護兩套系統?**

buildmoat.org



核心思想：把所有資料保存在 Kafka 裡（設定足夠長的保留期，例如 90 天）。當業務邏輯改變、或需要重新計算歷史資料，就用新版本的串流程式從 Kafka 最早的 offset 重播一遍，相當於做了一次「批次計算」。計算完成後，把流量切換到新版本。

Kappa 的優點：架構更簡單，只有一套邏輯需要維護。**Kappa 的缺點**：對重播大量歷史資料的場景，效能不如 Spark 的批次處理；Kafka 長期保留大量資料的儲存成本也更高。

> **在面試中怎麼選**：現代公司越來越傾向 Kappa，因為維護兩套系統的成本太高。但如果題目強調「需要跑超大規模的歷史分析（TB 級以上）」，Lambda 或純批次的方案更合理。先問清楚業務的延遲需求，再決定架構。

ETL 與 ELT

資料管線的另一個核心概念是資料轉換的時機：你是先轉換再載入，還是先載入再轉換？

ETL（Extract-Transform-Load）：先從源頭抽取（Extract）資料，在中間層做轉換（Transform）清洗，最後載入（Load）到目的地。傳統的資料倉儲方式，轉換邏輯在進入倉儲之前就完成。

ELT（Extract-Load-Transform）：先把原始資料直接載入到強大的分析型資料庫（如 BigQuery、Snowflake），再用 SQL 或框架在裡面轉換。現代雲端資料倉儲的計算能力強大到可以在存儲層直接做轉換。

buildmoat.org

ELT 的好處是保留了所有原始資料，轉換邏輯可以隨時修改重跑，且充分利用現代雲端資料倉儲的彈性算力。現在大多數新系統都傾向 ELT。

資料去哪裡？Data Warehouse vs Data Lake

處理後的資料需要一個地方存放，讓分析師或下游系統消費。

Data Warehouse（資料倉儲）

用於存放**結構化、已清洗的資料**，針對分析查詢優化。代表產品：BigQuery、Snowflake、Redshift。

特點：Schema-on-write（寫入前要定好結構）、查詢速度快、適合 BI 報表和 SQL 分析。

Data Lake（資料湖）

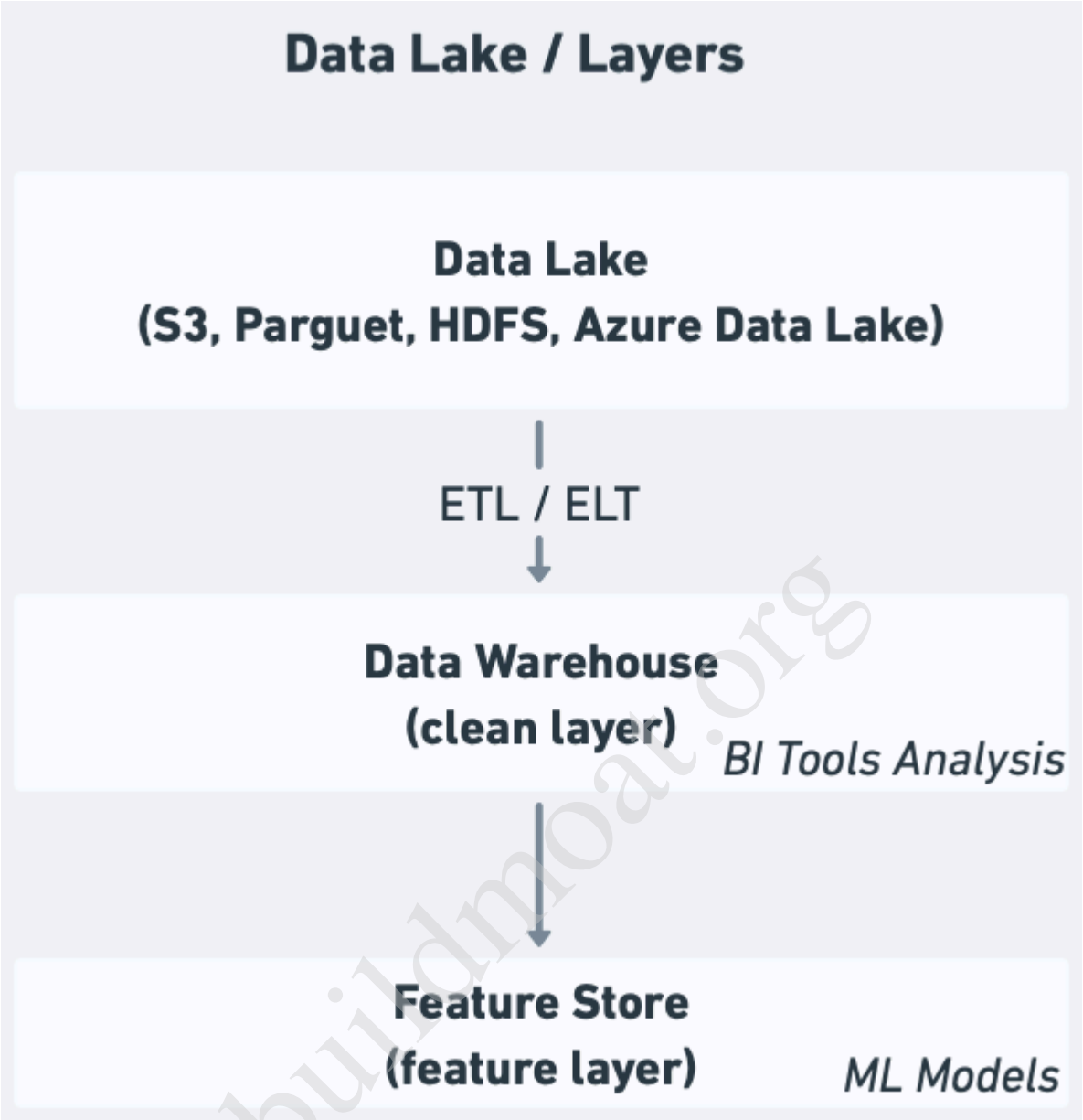
用於存放**原始、未處理的資料**，無論是結構化（CSV、Parquet）、半結構化（JSON、log）還是非結構化（圖片、影片）。代表技術：S3 + Parquet、HDFS、Azure Data Lake。

特點：Schema-on-read（讀取時才決定結構）、儲存便宜、彈性高、適合 ML 訓練和探索性分析。

Data Lakehouse（湖倉一體）

近年出現的架構，試圖結合兩者優點：用低成本的物件儲存（S3）存放資料，但透過 Delta Lake、Apache Iceberg 等格式層，在其上支援 ACID 事務、Schema 演化、和高效的分析查詢。Databricks、Snowflake 都在往這個方向走。

buildmoat.org



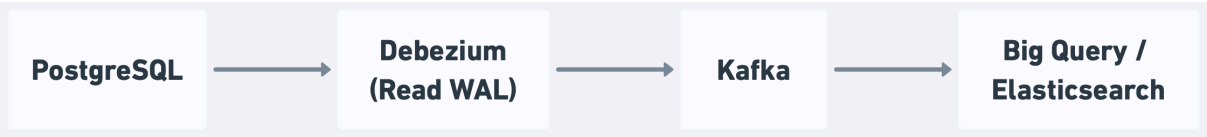
buildmoat.org

三個常見的管線模式

Change Data Capture（CDC，變更資料捕捉）

你的 App 把交易資料寫到 PostgreSQL，但分析師需要在 BigQuery 裡查詢它。最笨的做法是定期全量複製——每小時把整張表 dump 出來再匯入。這太慢、太浪費。

CDC 是更好的解法：監聽資料庫的 WAL（Write-Ahead Log），把每一筆 INSERT、UPDATE、DELETE 作為事件捕捉下來，實時同步到下游系統。



Debezium 是最常用的 CDC 工具，支援 PostgreSQL、MySQL、MongoDB 等主流資料庫。它把資料庫的每一個變更轉成一個 Kafka 事件，下游可以訂閱這些事件做各種用途：同步到分析倉儲、更新搜尋索引、讓快取失效、觸發業務流程。

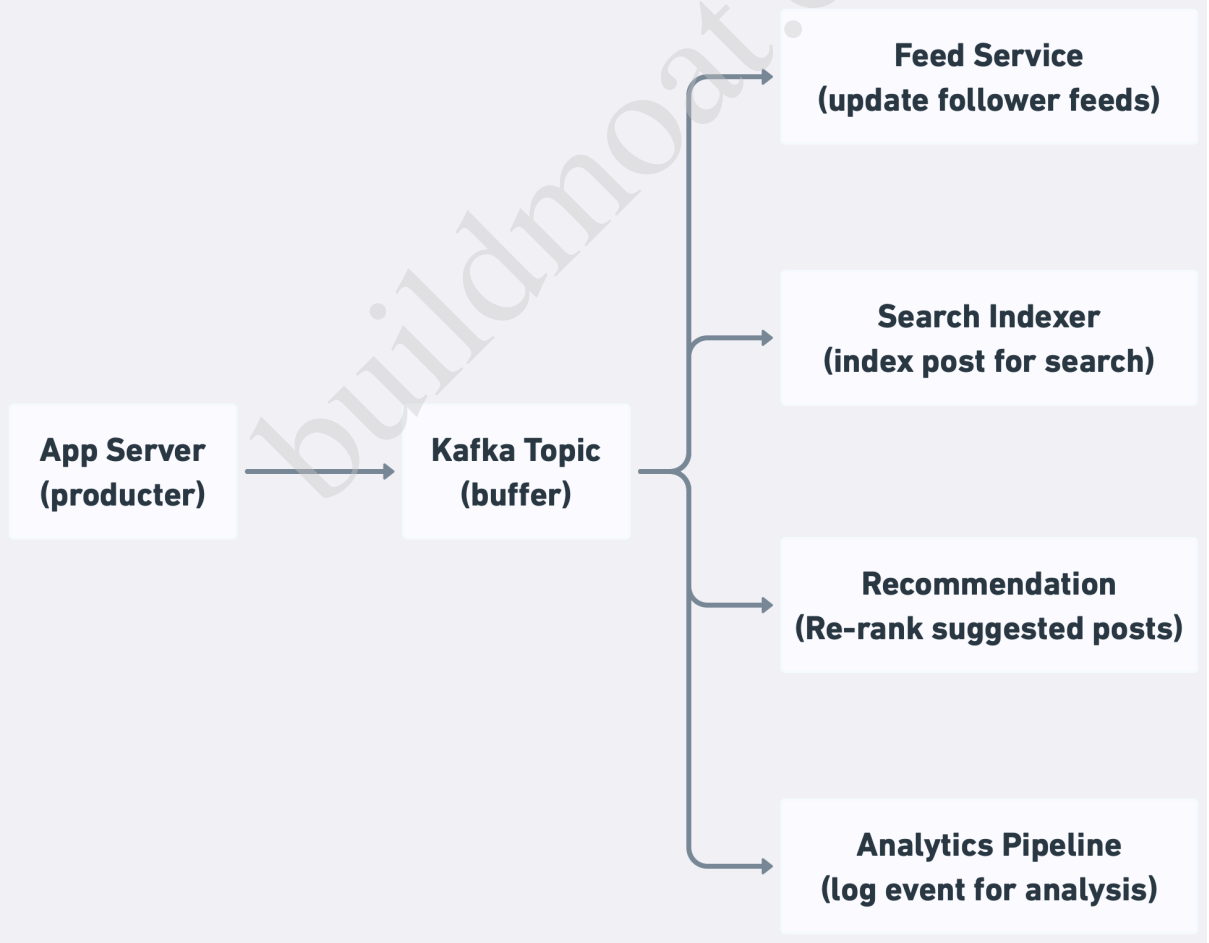
在面試中，當你需要在不影響線上 DB 效能的情況下把資料同步到另一個系統，CDC + Kafka 是標準答案。

Fan-out Pipeline（扇出管線）

一個上游事件，需要觸發多個下游處理。用戶發了一篇文章，需要：更新作者的文章列表、把文章推送到粉絲的 feed、更新搜尋索引、觸發 ML 模型重新排序推薦、記錄到分析系統。

最差的做法是 App server 直接呼叫所有下游服務——這讓上下游強耦合，任何一個下游服務慢或壞，整個發文就卡住了。

正確的做法是 **Kafka 的 pub/sub 模式**：App server 只發一個事件到 Kafka。每個下游服務各自訂閱這個 topic，獨立消費，互不影響。



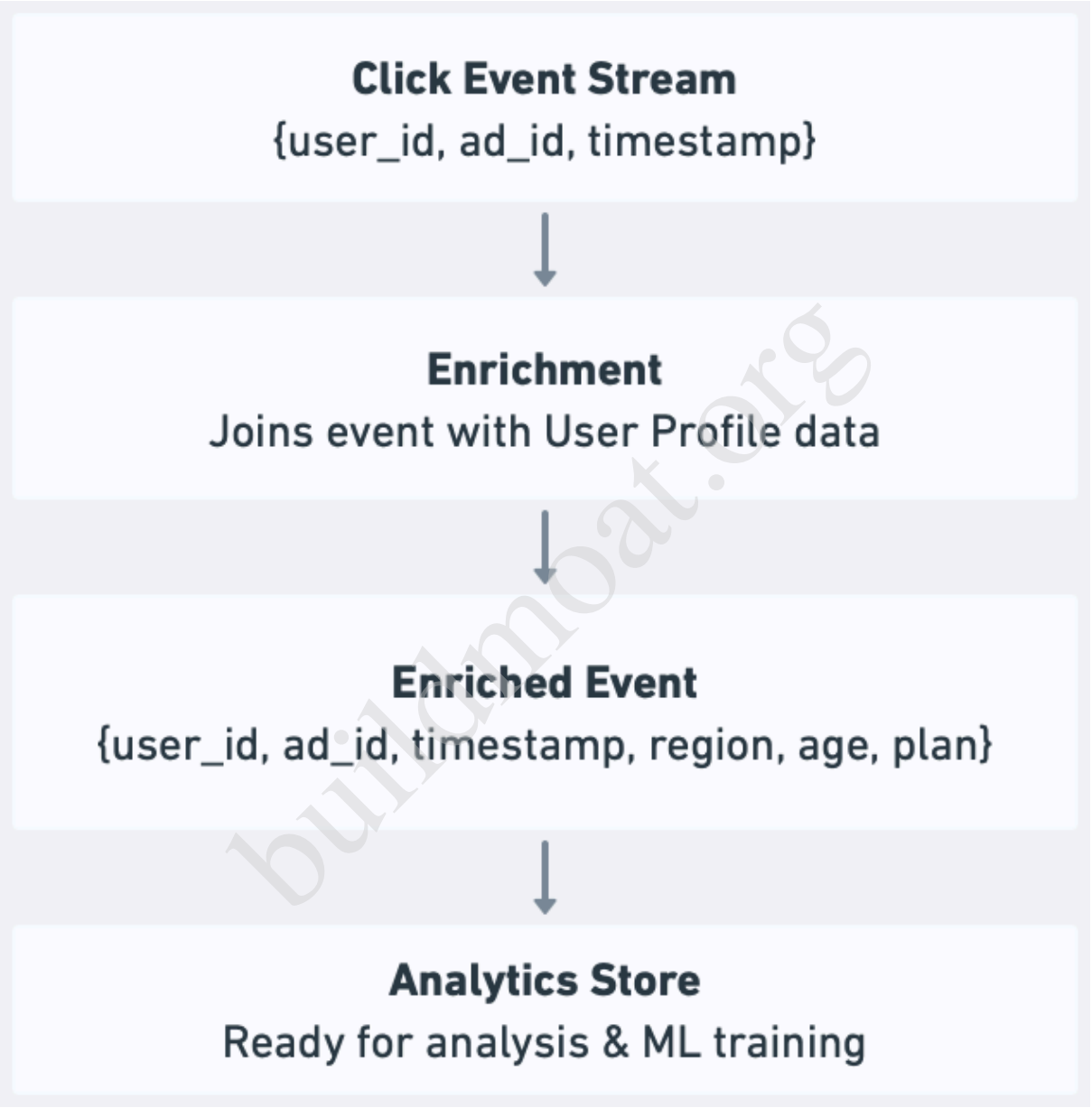
buildmoat.org

這讓上下游完全解耦，新增下游服務只需要訂閱 topic，不需要改動 App server 程式碼。

Data Enrichment Pipeline（資料豐富化管線）

很多分析需要把多個來源的資料合併。用戶點擊事件只有 user_id，但分析師需要知道這個用戶的地區、年齡、訂閱方案。

做法是在資料流過管線時，動態從 side input（旁路輸入）補充資訊：



buildmoat.org

注意：在高吞吐量的串流管線裡，每筆事件都去查資料庫會成為嚴重瓶頸。實務上會在本地維護一個熱點資料的快取（例如把用戶 profile 存在 Redis 或記憶體裡），並定期更新。

容錯機制

Data pipeline 最怕的不是速度慢，而是資料搞錯了，資料重複計算了兩次、還是有一批資料沒被處理到？這些問題比效能問題更難察覺，也更難修復。

三種語意保證

At-most-once（最多一次）：每筆資料最多處理一次，可能遺失。實作最簡單，但遺失資料通常無法接受。

At-least-once（至少一次）：每筆資料至少處理一次，可能重複。這是大多數系統的預設保證。遇到失敗就重試，但重試可能導致重複。

Exactly-once（恰好一次）：每筆資料精確地處理一次，不遺失、不重複。這是最強的保證，但代價也最高，需要框架和下游系統的協作。

> **實務上**：Exactly-once 很難完美實現，且有效能代價。很多系統選擇 at-least-once，再讓下游系統做**冪等處理（idempotency）**，即使同一筆資料進來兩次，結果也一樣（例如用 upsert 代替 insert）。

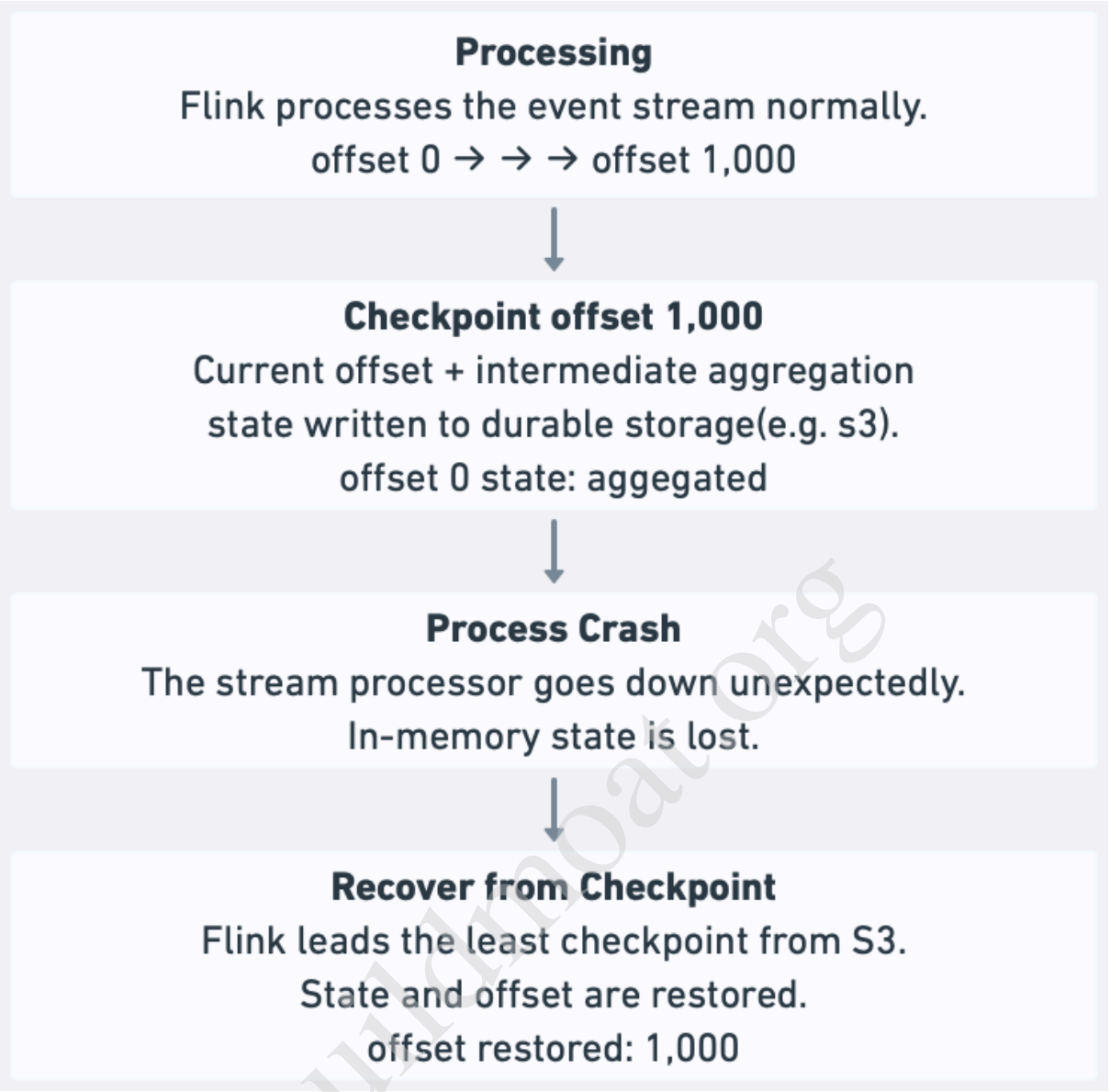
Checkpointing（檢查點）

串流處理器定期把當前的處理狀態（已處理到哪個 offset、當前的聚合中間結果）寫到持久化儲存（例如 S3）。這個快照叫做 checkpoint。

如果處理器崩潰，重啟後從最近的 checkpoint 恢復，從那個 offset 繼續消費，而不需要從頭來過。

Checkpoint 的頻率是個取捨：太頻繁，I/O 開銷大影響吞吐量；太少，崩潰後要重新處理的資料量就多。

buildmoat.org



buildnoat.org

冪等寫入 （Idempotent Writes）

即使有 at-least-once 保證，只要寫入是冪等的，重複的事件就不會造成問題。

```
-- 非冪等：重複執行會多插入一行
INSERT INTO events VALUES (user_id, event_type, timestamp);

-- 冪等：重複執行結果相同
INSERT INTO events VALUES (user_id, event_type, timestamp)
ON CONFLICT (event_id) DO NOTHING;
```

資料管線的最佳實踐：**pipeline 本身保證 at-least-once，下游寫入操作設計成冪等。** 這比實現 exactly-once 更簡單、更有彈性。

系統設計面試中怎麼談 Data Pipeline

Data Pipeline 通常在設計分析系統、推薦系統、監控系統、或任何「需要從大量資料中提取洞察」的題目時出現。

第一步：先問清楚延遲需求

最重要的問題是：**這個資料需要多快被看到？**

- 「幾秒內要看到」→ 串流處理
- 「幾分鐘可以接受」→ 微批次（Spark Streaming 或 Flink，視窗設短一點）
- 「幾小時或隔天」→ 批次處理

不要假設所有資料都需要即時，也不要不問就直接說「用 Kafka + Flink 做串流」。把這個問題丟出去，再根據答案選擇架構。

第二步：識別資料來源和目的地

明確說出資料從哪裡來、要到哪裡去：

「我們的來源是 PostgreSQL（交易資料）和 App server log（用戶行為事件）。目的地是 BigQuery（給分析師跑報表）和 Redis（給 API 即時查詢排行榜）。」

第三步：說明轉換邏輯的複雜度

簡單的過濾和聚合，用 Kafka + 簡單的 consumer 就夠了。複雜的 join、視窗聚合、ML feature 計算，才需要 Flink 或 Spark。

「我們需要把點擊事件和用戶 profile join 起來，然後以 5 分鐘為視窗計算每個廣告的 CTR。這需要串流框架的 windowing 支援，我會用 Flink。」

第四步：主動說明容錯策略

「處理器崩潰時，我們透過 Kafka offset 保證 at-least-once，並讓寫入 BigQuery 的操作冪等（用 event_id 做 deduplication），實際上達到 exactly-once 的效果。」

常見面試情境

分析系統（Analytics Dashboard）：「過去 24 小時的 GMV 圖表」可以用批次處理；「即時訂單數」需要串流。通常兩個都要，這時 Lambda Architecture 或「批次 + 小視窗串流」的混合方案是合理答案。

推薦系統：Feature engineering 通常是批次的（每天跑一次），但「用戶剛剛點了什麼」這種 real-time feature 需要串流更新。

buildmoat.org

監控告警系統：Log 收集是串流，異常偵測（例如 p99 延遲突然飆升）需要即時視窗聚合，一定是串流。

搜尋索引更新：資料庫變更透過 CDC 捕捉，推送到 Kafka，搜尋引擎（如 Elasticsearch）訂閱後即時更新索引。

常見的 Deep Dive 問題

「如果管線中斷了幾個小時，恢復後怎麼處理積壓的資料？」

這叫做 **backlog 消化**問題。幾個策略：

多開 consumer：Kafka 的 partition 數決定了最大並發消費數。如果 topic 有 100 個 partition，最多可以同時跑 100 個 consumer 加速消化積壓。

用批次補跑：如果 Kafka 保留了所有積壓的資料，也可以起一個獨立的 Spark 批次 job，專門跑那段時間的資料，和串流管線並行消化。

優先級：如果積壓了好幾天的資料，考慮先處理最新的資料（確保當前結果的及時性），再回頭補舊的。

「如何保證管線輸出的資料品質？」

資料品質問題很真實：上游改了 Schema、有 null value 出現在不該有的欄位、某個數值異常大或小。

Schema Registry：用 Confluent Schema Registry 管理 Kafka 事件的 schema，producer 發布的事件必須符合 schema，不合規的事件自動被拒絕。

資料品質監控：在管線輸出端設置自動化檢查：欄位的 null rate 超過閾值就告警、數值分佈超出歷史範圍就告警。工具有 dbt（data build tool）、Great Expectations 等。

死信佇列（DLQ）：解析失敗或不符合格式的事件，不要讓管線崩潰，送到 DLQ 隔離起來，人工調查原因後再決定如何處理。

「如何做 Schema 演化（Schema Evolution）？」

上游系統加了一個欄位，下游要怎麼處理？

向後相容（Backward compatible）的改動：新增欄位、讓欄位可選。舊版消費者讀到有新欄位的資料，直接忽略即可。

向前相容（Forward compatible）的改動：移除欄位時，新版消費者讀到沒有這個欄位的舊資料，需要有預設值。

buildmoat.org

用 **Apache Avro 或 Protobuf** 格式，配合 Schema Registry，可以強制執行相容性規則，讓 schema 變更安全可控。

總結

Data pipeline 的核心決策是**批次 vs 串流**，這個選擇由業務對延遲的容忍度決定。批次處理吞吐量高、成本低、適合歷史分析；串流處理延遲低、適合即時場景，但複雜度更高。

在架構上，**Kappa Architecture**（全串流 + Kafka 重播）是現代系統的主流選擇，因為它只需要維護一套邏輯。**Lambda Architecture**（批次 + 串流雙管線）在需要超大規模歷史計算時仍有一席之地。

三個最常出現在面試的模式：**CDC** 用於資料庫變更的即時同步、**Fan-out** 用於一個事件觸發多個下游處理、**Data Enrichment** 用於在管線中動態補充欄位。

容錯方面，記住一個實用的組合：**管線保證 at-least-once + 下游寫入設計成冪等**，比追求 exactly-once 更簡單也更可靠。

在面試中，先問清楚延遲需求，再選架構，再說明容錯策略。不要上來就說「用 Kafka + Flink」， 請表現出你是從需求推導技術選擇，而不是把技術砸進去。

buildmoat.org